Typesetting documents with AI

A written house style, a print stylesheet and a page-by-page review make AI drafts read like designed documents

AUTHORS	DATE	DOCUMENT
WindBorne Systems	September 2026	Example whitepaper · Draft

An AI model asked for a whitepaper will write a web page: colored boxes, bold phrases, bullet lists, a legend on every chart and a stock vocabulary readers now recognize at a glance. The fix is not a better prompt but a better starting point. We give the model three things: a stylesheet that makes the printed page look right by default, a written guide that says what good looks like for text, tables and figures, and a loop in which the model prints the document and looks at every page before calling it done. This paper describes that method and is itself set with it. We ask that teams writing papers, reports and proposals start from this template and send back what it gets wrong.

Why the defaults look wrong

Language models learned layout mostly from the web, so their instinct is to make a document scannable on a screen. Printed documents work differently. A reader of a proposal or a technical report reads it through, often on paper, and judges it partly by whether it looks like the work of someone careful. Color used as decoration, headings on every paragraph and a pull quote that repeats the text all signal the opposite.[1]

The writing has the same problem. Models reach for inflated words and fixed rhythms: three adjectives where one would do, a closing sentence that restates the paragraph, and hedges stacked on hedges. Each habit is small. Together they tell the reader that nobody weighed the words, and the reader discounts the claims along with the prose.

The method

The method has five steps, shown in Figure 1. The model first settles who the reader is, what they should do afterward and the three or four points that get them there. It then writes the document in plain HTML using only the classes in `style.css`, prints it to PDF with headless Chrome, and looks at an image of every page. Most of the quality comes from the fourth step, and from repeating it: a model that inspects its own pages finds stranded headings, colliding labels and half-empty pages that it cannot see in the source.

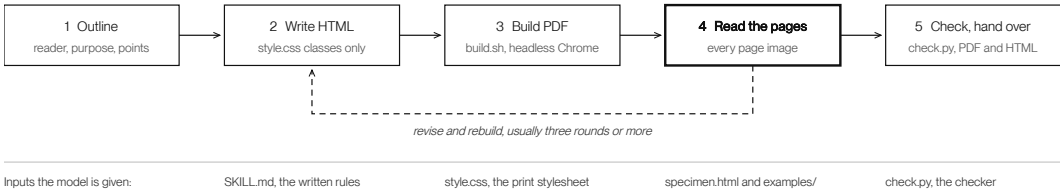


Figure 1. The five steps of the method. The heavier outline marks the step that does most of the work. The dashed path is the revision loop, which runs until a full read of the pages turns up nothing to fix.

The model starts from a complete example rather than a blank file. It copies structure from `examples/` and individual styles from the specimen, which shows each style once. Anything the stylesheet does not provide, such as a new color or a font size, is treated as a sign that the document is drifting from the house style rather than as a gap to fill.

Figures are drafted, not illustrated

Figures carry the most visible difference between a designed document and a generated one. We draw every figure as inline SVG, which prints sharp and stays editable as text, and follow the conventions of engineering drawing: a small set of line weights, each with one job, gray fills and hatching instead of color, square corners, and labels placed directly on the drawing.[2] Figure 2 shows the difference labeling alone makes.

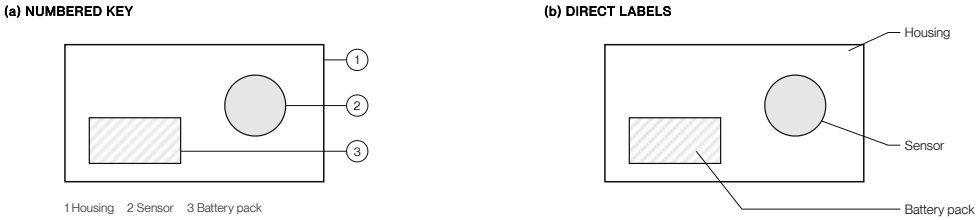


Figure 2. The same drawing labeled two ways. (a) A numbered key sends the eye back and forth between drawing and legend. (b) Direct labels on leader lines are read in place, and need no extra marks on the drawing.

Type in a figure must print at a size close to the caption. Because the drawing scales with the page, we fix the width of the drawing space and derive sizes from it. The printed size of one drawing unit is

$$s = \frac{W}{V} \tag{1}$$

where W is the printed width of the figure in points and V is the width of the SVG `viewBox` in drawing units. A full-width figure on a letter page with 0.8 in side margins has $W = 496.8$ pt, and with $V = 700$ each unit prints at 0.71 pt. A label set at font size f in drawing units then prints at

$$f_{\text{print}} = sf \approx 0.71 f \tag{2}$$

so the house label size of 8.6 prints at about 6.1 pt and the smallest permitted, 7.4, at about 5.3 pt. Line weights scale the same way. Appendix A lists them.

Checking the result

A written guide is easy to follow loosely, so the template includes a checker. It reads the HTML source for color, stray fonts, unnumbered or uncited figures, straight quotes, hyphens in ranges and a list of stock phrases, and reads the printed PDF for color, headings stranded at the foot of a page and a nearly empty last page. It runs in a second after each build:

```
./build.sh examples/whitepaper.html
python3 check.py examples/whitepaper.html
```

The checker catches mechanical faults, not judgment. In our trials the number of faults a model found in its own pages fell quickly over the first three rounds of the loop and then leveled off, as in Figure 3. What remained were matters of emphasis and wording that a person had to call.

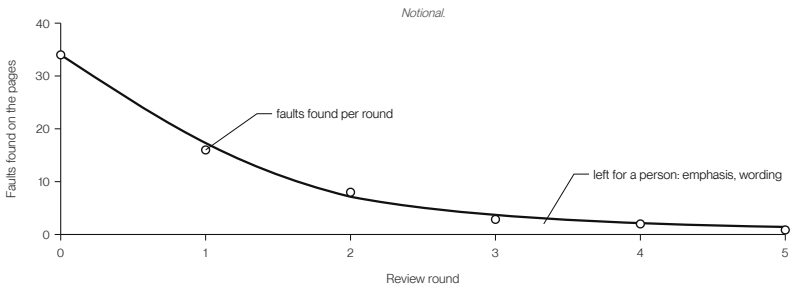


Figure 3. Faults a model finds by reading its own printed pages, by review round. Most mechanical faults are gone after three rounds. The data are illustrative of the pattern, not a measurement.

Choice of format

We write in HTML and CSS rather than a word processor, LaTeX or slide software. Table 1 gives our assessment. HTML is the format models write most reliably, it holds vector figures directly in the source, and it produces a text file whose changes can be reviewed line by line. Its weakness is print control: Chrome supports page size, margins, footers and page-break rules, but not every feature of a typesetting system, such as balanced columns or true footnotes at the foot of each page.[3] For papers and whitepapers of the kind we write, that trade is worth making.

Format	Print control	Written reliably by AI	Easy hand edits	Vector figures in source	Readable diffs
Word processor	◑	◔	●	◔	○
LaTeX	●	◑	◔	◑	●
Slide software	◔	◔	●	◑	○
HTML and CSS	◕	●	◑	●	●

Table 1. Formats for AI-written print documents, rated from empty (poor) to full (good). The ratings are our judgment from use, not a benchmark. HTML trails LaTeX only on fine print control.

Limits

The method makes documents consistent. It does not make them right. A model following the guide will still state a number it cannot support unless it is told not to, so the guide requires a source or an explicit label for every figure a reader could question, and a visible placeholder where a fact is missing. The checker’s phrase list is

short and will need to grow. The grayscale rule rules out color maps, which some scientific figures need. A document that depends on them should say so and keep color to those figures alone.

What we ask

Teams writing a paper, report or proposal should give their AI the `SKILL.md` guide and this repository, and send back the first document they make with it together with the places where the guide was silent or wrong. We will fold those into the guide and the stylesheet, so that the next document starts from a better default.

1. Robert Bringhurst, *The Elements of Typographic Style*, version 4.0, Hartley & Marks, 2012.
2. American Society of Mechanical Engineers, *Line Conventions and Lettering*, ASME Y14.2-2014, 2014.
3. W3C, “CSS Paged Media Module Level 3,” Working Draft, *www.w3.org/TR/css-page-3*.

Appendix A. Figure conventions at a glance

Table 2 lists the line weights and type sizes from the guide with the printed size each gives on a full-width letter figure, from equation (1). Edward Tufte's advice to spend ink on data and little else is a good test for anything not listed here.[4]

Element	Use	Drawing units	Printed, pt
Hairline	Leaders, dimensions, ticks	0.4	0.28
Hatching	Cut surfaces, set-apart regions	0.5	0.35
Outline	Boxes, parts, axes, connectors	0.8	0.57
Emphasis	Section outlines, key edges	1.2	0.85
Heavy	The subject of the figure	1.6	1.14
Label	Names of parts and series	8.6	6.11
Secondary text	Descriptions, tick labels	8.0	5.68
Smallest text	Notes such as units	7.4	5.26

Table 2. Line weights and type sizes for figures. Printed sizes assume a viewBox 700 units wide across a 496.8 pt measure.

4. Edward R. Tufte, *The Visual Display of Quantitative Information*, 2nd ed., Graphics Press, 2001.